Course: Speech Recognition and Generation course
Assignment: Automatic Speech Recognition — Russian Numbers from Scratch
Author:
University: ITMO, AITH
Date: May 2025

---

1. Introduction

In this assignment, I've tried (pretty much desperately) to train an automatic speech recognition (ASR) system from scratch to recognize spoken Russian numbers in the range from 1,000 to 999,999.

The model is trained on a provided dataset of audio-transcription pairs, where transcriptions are numeric values and audio varies in format and sample rate. Unlike previous assignments, pre-trained weights are not allowed — the model must be developed and trained entirely from scratch using only the provided training data.

The main goal of the project is to achieve the lowest possible character error rate (CER) while adhering to a strict parameter constraint (≤5M) and ensuring generalization across different speakers, genders, and out-of-domain voices.

Throughout the project, I experiment with different model architectures, decoding strategies, and audio augmentation techniques. I also implement normalization/denormalization for the spoken form of numeric transcriptions, and evaluate the model using average CER.

Being honest,

i do realize the work is far from great. It seems I failed to manage my priorities and time properly(or, even I've managed them well but this group project was far from the top of this chain...) I'm really sorry 'cause the activity is truly interesting and valuable. I'll experiment with this later for myself only but today we have what we have 😕

---

2. Setup and Tools

All experiments were implemented in Python using the following frameworks and tools:

- `PyTorch` and `torchaudio` for model training and audio processing
- `num2words` for normalization/denormalization of numbers
- `KenLM` and `pyctcdecode` for language model integration
- `Google Colab Pro` and `Kaggle` with GPU runtime for training

Target input format is 16 kHz mono audio. All non-conforming samples were resampled.

The dataset consists of audio files paired with numeric transcriptions in the range from 1,000 to 999,999. To prepare the data for training, the following preprocessing steps were applied:

- **Metadata loading**: Each `.csv` file (for train and validation) is parsed to extract the path, transcription, and speaker ID.
- **Transcription normalization**: Numeric transcriptions are converted into their full Russian verbal form using the `num2words` library (`lang="ru"`), and commas are removed for consistency.
- **Vocabulary creation**: A character-level vocabulary is built dynamically from the training transcriptions. A `<PAD>` token is added and assigned index 0.
- **Mel-spectrogram extraction**: Audio waveforms are converted to log-Mel spectrograms using 80 Mel bins (`n_mels=80`, `n_fft=400`, `hop_length=160`).
- **Audio augmentation**: During training, audio is randomly augmented with volume shifts, frequency masking, and time masking (specifically: `Vol`, `FrequencyMasking`, and `TimeMasking`).
- **Tokenization**: Normalized text transcriptions are converted to integer token sequences using the `char2idx` mapping.
- **Speaker metadata**: Each training example retains its `spk_id`, which is used for speaker-level error analysis.
- **Length-based bucketing (for Conformer)**: For training the Conformer model, a custom `BucketBatchSampler` was used to group samples of similar input lengths into the same batch. This minimizes padding and improves training efficiency, especially for variable-length audio inputs. Buckets were defined using input length boundaries (`[100, 150, 200, 300, 500]`), and batches were optionally shuffled within and across buckets.

This preprocessing pipeline ensures the model operates on consistent, augmented 16 kHz spectrograms and aligns with the dataset's structural constraints.

---

4. Models

CNN-based model

As a baseline, I implemented a lightweight CNN-based encoder architecture for ASR. The model was designed with the goal of staying within the constraint of 5 million trainable parameters (actual count: **5,058,069**).

Architecture

The model is composed of the following components:

- **Input**: Log-Mel spectrograms with 80 Mel bands (`in_channels=80`)
- **Convolutional stack**:
  - 5 stacked 1D convolutional layers with increasing channel dimensions (512 → 768 → 512)
  - Each convolution is followed by `BatchNorm1d` and ReLU activation
  - A dropout layer (`p=0.2`) is applied after the final convolution block
- **Classifier**:
  - A single linear projection layer maps the final hidden state to character logits (`Linear(hidden_dim, num_classes)`)

The output of the model is passed through a `log_softmax` function and used with the CTC loss during training.

Key Properties

- Fully convolutional encoder without recurrence or attention
- Efficient for short utterances like spoken numbers
- Keeps parameter count just below the 5M threshold
- Suitable for real-time or embedded inference due to low latency

This CNN encoder serves as a strong baseline before experimenting with more expressive architectures like Conformer.

Training Configuration

- **Loss**: Connectionist Temporal Classification (CTC) loss with `blank=0` and `zero_infinity=True`
- **Optimizer**: Adam optimizer with a learning rate of `1e-3`

Conformer-based Model

To improve upon the CNN baseline, I implemented a compact version of the Conformer architecture — a hybrid encoder combining convolutional and self-attention modules.

The final model contains **4,726,133** trainable parameters, remaining within the 5M limit.

Architecture

The model is composed of the following key components:

- **Convolutional subsampling**: A `ConvSubsampling` block reduces the input sequence length by a factor of 4 using two 2D convolutions with stride=2. This reduces computational cost and enables faster training.
- **Stacked Conformer blocks**:
  - Each `ConformerBlock` contains:
    - Two positionwise `FeedForwardModule` blocks (with expansion factor = 2)
    - Multi-head self-attention (`MultiheadAttention`, `n_heads=8`)
    - Depthwise separable convolution module (`ConvModule`) with GLU gating and `SiLU` activation
    - Residual connections and `LayerNorm` after each sub-block
  - A total of 6 Conformer blocks were stacked (`num_layers=6`)
- **Output projection**: A linear layer projects the encoder output to character logits (`Linear(dim=224, num_classes)`)

Key Properties

- Combines local convolutional modeling with global attention
- Improves robustness to variable speech rate and speaker variation
- Subsampling reduces input length by 4×, improving efficiency
- Residual and normalization layers promote stable training

Training Configuration

- **Loss**: CTC loss with `blank=0` and `zero_infinity=True`
- **Optimizer**: Adam optimizer with a learning rate of `1e-3` and `weight_decay=3e-4` to regularize training which was being changed during the little exps

5. Results

CNN-based Model

The CNN model was trained for 15 epochs using the CTC loss and Adam optimizer.

Best Result

- **Best validation loss** was achieved at **epoch 4**, with:
  - **Train Loss**: 1.6601
  - **Val Loss**: 2.2956

After epoch 4, validation loss began to increase despite continued improvements in training loss, indicating the start of overfitting.

Conformer-based Model

I conducted three main training experiments for the Conformer model, exploring different dropout rates and regularization strategies. Early stopping was used with a patience of 3 or 4 epochs depending on the run.

Training Details

Try 1 (aka Best Result)

- **Avg Train Loss** dropped rapidly (e.g., to ~0.02 by epoch 5)
- **Best validation loss**: **0.600**
- Training was accidentally run for 20 epochs instead of 10
- Still achieved the best performance despite lack of scheduler

Try 2

- Increased dropout to 0.3
- Early stopping was triggered at epoch 4
- Best validation loss: 0.6853 (epoch 1), followed by degradation

Try 3

- Added learning rate scheduler (halves LR on plateau)
- Best validation loss: 0.8242 (epoch 2)
- Early stopping at epoch 5 after overfitting

Conclusion

The best-performing model was trained in **Try 1**, where a relatively low dropout and moderate weight decay produced strong generalization to the validation set. Further regularization or learning rate scheduling in Try 2 and Try 3 did not lead to improvement and may have underfitted or destabilized training early.

General Conclusion

In contrast to the CNN baseline (best val loss ≈ 2.29), the Conformer achieved significantly lower loss (best ≈ 0.60), confirming its superior modeling capabilities for this task.

6. Decoding and Inference

CNN-based Model

For the CNN model, I experimented with several decoding strategies based on the CTC output:

1. Greedy Decoding (`argmax`)

- The simplest approach: select the most probable token at each time step.
- Resulted in highly noisy outputs filled with `<PAD>` tokens.
- Does not handle repeated characters or blank tokens correctly.
- Example output:
  `<PAD>д<PAD>тдпд...<PAD>`

2. Custom Beam Search (without LM)

- Implemented a custom beam search decoder with `beam_width=10`.
- This approach maintained top sequences over time and avoided repeated predictions.
- Significantly improved over greedy decoding, but still produced many non-words and inconsistent letter groupings.
- Sample outputs:
  `ОСЕТСДЯТ БСТ СТЬ`, `ТЕТ ТЕСЯСЯТЬС`

3. Inference Pipeline

- The model was evaluated on `val_loader` in evaluation mode (`torch.no_grad()`).
- Predictions were obtained from the model as log-probabilities and decoded using beam search.
- Inference was implemented in batch mode with post-processing to extract character strings.

Summary

Decoder	Output Quality	Notes
Greedy	Poor	Overrun with `<PAD>` tokens
Custom Beam Search	Moderate	Cleaner outputs but unstable grammar
LM Fusion	Not used here	Tested later with Conformer

In conclusion, even with beam search, the CNN model struggled to produce fluent and interpretable text outputs. This emphasized the need for stronger sequence modeling (e.g., Conformer) and external language model support.

6. Decoding and Inference

Conformer-based Model

For the Conformer-based ASR model, I explored decoding and evaluation through two main approaches: direct character decoding and numeric post-processing. The aim was to recognize and transcribe spoken Russian numbers accurately.

1. Greedy Decoding

- The most straightforward decoding strategy: take the highest-probability token at each time step.
- Used a standard CTC decoding postprocessing: collapsing repeats and removing `<PAD>` (index 0).
- **CER (greedy)**: `0.1918`
  This result indicates relatively low character error in raw text, though it's not directly tied to numeric accuracy.

2. Fuzzy Number Matching with Correction and Conversion

- Each predicted sequence was decoded to a string, then mapped to a number using a custom fuzzy-matching pipeline:
  - Applied Levenshtein-based correction against a dictionary of known Russian number words.
  - Converted corrected strings to integers via rule-based parsing (units, tens, hundreds, thousands).
- This strategy allowed for numerical evaluation even in presence of minor transcription errors.
- Example:
  Predicted: `'восемьсот орок девят дтысдч девятьсот пять'`
  Parsed: `849905`
- **CER (fuzzy number decoding)**: `0.3712`
  Higher than raw character CER — likely due to decoding instability in numeric structure.

3. Language Model Fusion

- Tried to integrate KenLM via `pyctcdecode` using a character-level corpus and n-gram ARPA model.
- Although the LM was successfully compiled and loaded, it failed to improve predictions:
  - The training corpus was too small.
  - Outputs remained inconsistent or empty.
  - The LM pipeline was eventually abandoned in favor of simpler decoding.

4. Inference Strategy

- The best model (from experiment 1 with validation loss ≈ 0.6) was loaded in evaluation mode.
- Inference was run in batches using `torch.no_grad()`.
- Decoded predictions were matched against ground truth, both in raw text and numeric form.

Summary

Decoder	CER	Notes
Greedy (char-level)	0.1918	Clean character outputs, but no numeric understanding
Fuzzy Number Decoding	0.3712	Enabled numeric evaluation, but more sensitive to errors
Beam Search with LM (KenLM)	—	Attempted but not finished due to limited training data

Conclusion

The Conformer model significantly outperformed the CNN-based baseline in both decoding quality and numerical understanding. While language model fusion was unsuccessful due to data scarcity(or the author's lack of knowledhe, more likely), even the simple greedy decoding followed by fuzzy numeric mapping produced usable outputs. This architecture demonstrates better temporal modeling and generalization to number transcription tasks.

7. Conclusion

This work compared two ASR architectures — a CNN-based baseline and a Conformer — on the task of recognizing spoken Russian numbers. While the CNN model struggled with stable outputs and failed to generalize, the Conformer achieved significantly better performance, even under greedy decoding. Attempts to integrate a language model were limited by data size(and not only haha), but the overall results confirmed that modern sequence models like Conformer are far more suitable for structured speech tasks such as number transcription.